

animations

July 14, 2025

```
[ ]: from manim import *
```

```
[ ]: config.media_width = "80%"  
config.verbosity = "WARNING"
```

1 Ein/Ausblendeanimationen

```
[ ]: %%manim -qm -v CRITICAL CreateExample  
class CreateExample(Scene):  
    def construct(self):  
        square = Square(fill_color=BLUE, fill_opacity=0.5)  
        self.play(Create(square))  
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL WriteExample  
class WriteExample(Scene):  
    def construct(self):  
        text = Text("Hallo Welt")  
        self.play(Write(text))  
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL AddTextLetterByLetterExample  
class AddTextLetterByLetterExample(Scene):  
    def construct(self):  
        text = Text("Hallo Welt")  
        self.play(AddTextLetterByLetter(text))  
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL DrawBorderThenFillExample  
class DrawBorderThenFillExample(Scene):  
    def construct(self):  
        square = Square(fill_color=BLUE, fill_opacity=0.5).shift(2*LEFT)  
        square2 = Square(fill_color=BLUE, fill_opacity=0.5).shift(2*RIGHT)  
        self.play(DrawBorderThenFill(square))  
        self.wait()  
        self.play(Create(square2))  
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL IncreasingSubsetsExample
class IncreasingSubsetsExample(Scene):
    def construct(self):
        dots = VGroup(*[Dot(point=[x, 0, 0]) for x in range(-5, 6)])
        self.play(SpiralIn(dots))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL ShowSubmobjectsExample
class ShowSubmobjectsExample(Scene):
    def construct(self):
        text = Text("Hallo Welt")
        self.play(ShowSubmobjectsOneByOne(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL SpiralExample
class SpiralExample(Scene):
    def construct(self):
        text = Text("Hallo Welt")
        self.play(SpiralIn(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL RemoveExample
class RemoveExample(Scene):
    def construct(self):
        obj = Circle()
        self.play(Create(obj))
        self.wait()
        self.play(Uncreate(obj))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL FadeExample
class FadeExample(Scene):
    def construct(self):
        obj = Circle()
        self.play(FadeIn(obj))
        self.wait()
        self.play(FadeOut(obj))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL GrowExample
class GrowExample(Scene):
    def construct(self):
        circ = Circle()
        self.play(GrowFromCenter(circ))
        self.wait()
        self.play(FadeOut(circ))
        self.wait()
```

```
self.play(GrowFromEdge(circ, DR), run_time=4)
self.wait()
```

2 Indications

```
[ ]: %%manim -qm -v CRITICAL WaveExample
class WaveExample(Scene):
    def construct(self):
        text = Text("Wellen")
        self.play(ApplyWave(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL BlinkExample
class BlinkExample(Scene):
    def construct(self):
        text = Text("Wellen")
        self.play(Blink(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL CircumscribeExample
class CircumscribeExample(Scene):
    def construct(self):
        text = Text("Wellen")
        self.add(text)
        self.play(Circumscribe(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL FlashExample
class FlashExample(Scene):
    def construct(self):
        text = Text("Wellen")
        self.add(text)
        self.play(Flash(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL FocusOnExample
class FocusOnExample(Scene):
    def construct(self):
        text = Text("Wellen")
        self.add(text)
        self.play(FocusOn(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL IndicateExample
class IndicateExample(Scene):
    def construct(self):
        text = Text("Wellen")
```

```
self.play(Indicate(text))
self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL PassingFlashExample
class PassingFlashExample(Scene):
    def construct(self):
        circle = Circle()
        self.add(circle)
        self.play(ShowPassingFlash(circle))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL WiggleExample
class WiggleExample(Scene):
    def construct(self):
        text = Text("Wellen")
        self.play(Wiggle(text))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL HomotopyExample
class HomotopyExample(Scene):
    def construct(self):
        square = Square()
        def wave(x, y, z, t):
            return x, y + 0.5 * np.sin(2 * np.pi * t + x), z
        self.play(Homotopy(wave, square))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL PathExample
class PathExample(Scene):
    def construct(self):
        path = Circle()
        dot = Dot()
        self.play(MoveAlongPath(dot, path))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL RotateVsAnimate
class RotateVsAnimate(Scene):
    def construct(self):
        square = Square()
        self.add(square)
        self.play(square.animate.rotate(PI / 3))
        self.wait()
        self.play(Rotate(square, PI / 3))
        self.wait()
```

```
[ ]: %%manim -qm -v CRITICAL TransformExample
class TransformExample(Scene):
```

```

def construct(self):
    square = Square()
    circle = Circle()
    self.add(square)
    self.play(TransformFromCopy(square, circle))
    self.wait()
    self.play(square.animate.shift(UP))
    self.wait()
    #self.play(circle.animate.shift(UP))
    #self.wait()

```

```

[ ]: %%manim -qm -v CRITICAL ReplacementTransformExample
class ReplacementTransformExample(Scene):
    def construct(self):
        square = Square()
        circle = Circle()
        self.add(square)
        self.play(ReplacementTransform(square, circle))
        self.wait()
        self.play(circle.animate.shift(UP))
        self.wait()

```

```

[ ]: %%manim -qm -v CRITICAL FadeTransformExample
class FadeTransformExample(Scene):
    def construct(self):
        square = Square().shift(2*LEFT)
        circle = Circle().shift(2*RIGHT)
        self.add(square)
        self.play(FadeTransform(square, circle))
        self.wait()

```

```

[ ]: %%manim -qm -v CRITICAL TexTransform
class TexTransform(Scene):
    def construct(self):
        #expr1 = MathTex(r"x+x")
        #expr2 = MathTex(r"2\cdot x")
        #expr1 = MathTex(r"y", "+", r"x").shift(UP)
        #expr2 = MathTex(r"2\cdot", "x")
        expr1 = MathTex(r"{{x}}+{{x}}")
        expr2 = MathTex(r"2\cdot {{x}}")
        expr1[1].set_color(YELLOW)
        self.add(expr1, index_labels(expr1))
        self.play(TransformMatchingTex(expr1, expr2))
        self.wait()

```

```

[ ]: %%manim -qm -v CRITICAL MovingFrameBox
class MovingFrameBox(Scene):

```

```

def construct(self):
    text=MathTex(
        r"\frac{d}{dx}f(x)g(x)=",
        r"f(x)\frac{d}{dx}g(x)",
        "+",
        r"g(x)\frac{d}{dx}f(x)"
    )
    self.play(Write(text))
    framebox1 = SurroundingRectangle(text[1], buff = .1)
    framebox2 = SurroundingRectangle(text[3], buff = .1)
    self.play(
        Create(framebox1),
    )
    self.wait()
    self.play(
        ReplacementTransform(framebox1,framebox2),
    )
    self.wait()

```

```

[ ]: %%manim -qm -v CRITICAL CorrectLaTeXSubstringColoring
class IncorrectLaTeXSubstringColoring(Scene):
    def construct(self):
        equation = MathTex(
            r"e^{\{x\}} = \{x\}^0 + \{x\}^1 + \frac{1}{2} x^2 + \frac{1}{6} x^3 + \color{red}{\dots}
            \color{red}{\rightarrow}\frac{1}{n!} x^n + \color{red}{\dots}"
        )
        equation.set_color_by_tex("x", YELLOW)
        self.add(equation)

class CorrectLaTeXSubstringColoring(Scene):
    def construct(self):
        equation = MathTex(
            r"e^x = x^0 + x^1 + \frac{1}{2} x^2 + \frac{1}{6} x^3 + \color{red}{\dots} + \color{red}{\rightarrow}\frac{1}{n!} x^n + \color{red}{\dots}",
            substrings_to_isolate="x"
        )
        equation.set_color_by_tex("x", YELLOW)
        self.add(equation, index_labels(equation))

```

```

[ ]: %%manim -qm -v CRITICAL IndexLabelsMathTex
class IndexLabelsMathTex(Scene):
    def construct(self):
        text = MathTex(r"\binom{2n}{n+2}", font_size=96)

        # index the first (and only) term of the MathTex mob
        self.add(index_labels(text[0]))

```

```
text[0][1:3].set_color(YELLOW)
text[0][3:6].set_color(RED)
self.add(text)
```

```
[ ]: %%manim -qm -v CRITICAL LaTeXMathFonts
class LaTeXMathFonts(Scene):
    def construct(self):
        tex = Tex(
            r"$x^2 + y^2 = z^2$",
            tex_template=TexFontTemplates.french_cursive,
            font_size=144,
        )
        self.add(tex)
```

```
[ ]: %%manim -qm -v CRITICAL LaTeXAlignEnvironment
class LaTeXAlignEnvironment(Scene):
    def construct(self):
        tex = MathTex(r'f(x) \&= 3 + 2 + 1\\ \&= 5 + 1 \\ \&= 6', font_size=70)
        self.add(tex, index_labels(tex))
```

```
[ ]:
```